LiquidHaskell: Experience with Refinement Types in the Real World

Niki Vazou, Eric L. Seidel, Ranjit Jhala

Szabó Márk

Haskell's Type System

Haskell is a **statically typed** language.

- Every expression has a type.
- Types are checked before the program runs.
- Types are often inferred automatically.

Basic Types

Some common built-in types:

```
Int        -- integer
Float      -- floating point number
Double     -- floating point number
Num        -- [Int, Float, Double, ...]
Bool       -- True or False
Char       -- single character
String     -- text
```

Type Annotations

The operator :: means “has type”.

```
age :: Int
age = 23

name :: String
name = "Mark"

active :: Bool
active = True
```

Function Types

The arrow (→) separates input and output types.

```
add :: Int -> Int -> Int
add x y = x + y
```

- First Int: first argument
- Second Int: second argument
- Last Int: result

Equivalent in a C-like language:

```
int add(int x, int y) { return x + y; }
```

Type Inference

Haskell can infer many types automatically.

```
1 age = 23
2 double x = x * 2
```

In GHCi we can inspect inferred types:

```
1 :t True
2 True :: Bool
3
4 :t "hello"
5 "hello" :: String
6
7 :t double
8 double :: Num a => a -> a
```

Limits of The Type System

We can’t define constraints for the types of a function like:

- Conditions about the value: `x > 0`
- Relationships between values: `a > b`

To check for positive we need to check inside the function:

```haskell
sqrtRoot :: Double -> Double
sqrtRoot x
  | x > 0 = sqrt x
  | otherwise = error "x must be greater than 0"
```

LiquidHaskell

Many of these properties and relationships can be verified using a combination of **Refinement Types** and **external SMT solvers**.

This paper introduces and experiments with **LiquidHaskell** a refinement typechecker used to verify more than **10.000** lines of Haskell code across popular libraries.

Including: *bytestring*, *text*, *vector-algorithms* and *xmonad*.

Refinement Types

Refinement types:

- contain a **predicate** which is assumed to be true for any element of the refined type.
- can express **preconditions** when used as function arguments or **postconditions** when used as return types.

Examples:

positive natural numbers

$x : \{n \in \mathbb{N} \mid n > 0\}$

even natural numbers

$x : \{n \in \mathbb{N} \mid n \bmod 2 = 0\}$

result is a positive number

$f : \mathbb{N} \to \{y \in \mathbb{N} \mid y > 0\}$

result > input

$f(x : \mathbb{N}) : \mathbb{N} \to \{y \in \mathbb{N} \mid y > x\}$

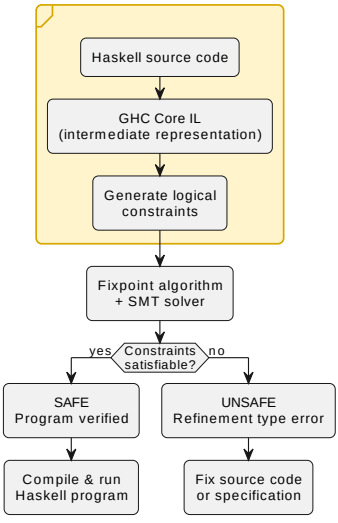
LiquidHaskell Architecture

Core

- Core is a simpler intermediate representation (GHC translates Haskell source to it)
- Source locations are preserved for error reporting

Constraints

- Refinement types are translated into logical constraints
- These constraints describe the properties that must hold



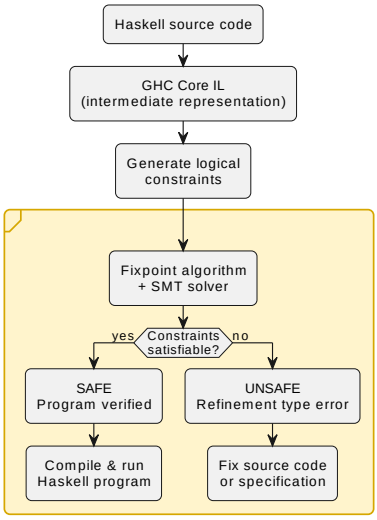
LiquidHaskell Architecture

Solution

- A fixpoint algorithm processes the constraints
- Any *SMT-LIB2* compatible solver checks whether the constraints can be satisfied (eg.: *Z3*, *CVC4*, …)

Result

- **SAFE** or **UNSAFE**: based on the verification of the refinement properties
- Errors are mapped back to their source-code locations



LiquidHaskell

LiquidHaskell is an **optional** type checker for Haskell. Refinement types are written inside **comments**, so they are optional and normal Haskell compilers can ignore them:

```
1 {-@ ... @-}
```

Execution

LiquidHaskell performs static verification before the Haskell program is executed. It does not become part of the executable or perform any checks during runtime.

LiquidHaskell Basics

```
1  {-@ type Natural = {v:Int | v >= 0} @-}
2  {-@ type Pos = {v:Int | v > 0} @-}
3
4  {-@ increment :: Natural -> Pos @-}
5  increment :: Int -> Int
6  increment x = x + 1
7
8  good1 = increment 0
9  good2 = increment 4
10
11 -- verification will fail here stating: Type Mismatch
12 bad = increment (-1)
```

LiquidHaskell Basics

```
1  {-@ type Natural = {v:Int | v >= 0} @-}
2  {-@ type Pos = {v:Int | v > 0} @-}
3
4  {-@ divide :: n:Natural -> d:Pos -> {v:Natural | v <= n} @-}
5  divide :: Int -> Int -> Int
6  divide x y = x `div` y
7
8  good = divide 5 2
9
10 -- With LH -> Type Mismatch before executing the code
11 -- Without LH -> runtime error: divide by zero
12 bad = divide 5 0
```

LiquidHaskell Parameterized Types

Type parameters are written as lowercase and **value parameters** are written as uppercase.

- **Type parameter:** *Int*, *Bool*, *[Int]*
- **Value parameter:** *0*, *10*, *100*

```
1 {-@ type NonEmpty a = {v:[a] | len v > 0} @-}
2
3 {-@ numbers :: NonEmpty Int @-}
4 numbers :: [Int]
5 numbers = []
6
7 {-@ letters :: NonEmpty Char @-}
8 letters :: [Char]
9 letters = ['a', 'b', 'c']
```

LiquidHaskell Parameterized Types

```
1  {-@ type Rng Low High = {v:Int | v >= Low && v <= High} @-}
2
3  {-@ grade :: Rng 0 100 -> Int @-}
4  grade :: Int -> Int
5  grade score
6      | score >= 90 = 5
7      | score >= 75 = 4
8      | score >= 60 = 3
9      | score >= 50 = 2
10     | otherwise   = 1
11
12 good = grade 85
13 bad = grade 120
```

LiquidHaskell Parameterized Types

```
1  -- a: element type, N list length
2  {-@ type ListN a N = {v:[a] | len v == N} @-}
3
4  {-@ numbers :: ListN Int 3 @-}
5  numbers :: [Int]
6  numbers = [1, 2, 3]
7
8  {-@ letters :: ListN Char 2 @-}
9  letters :: [Char]
10 letters = ['a', 'b']
11
12 {-@ invalid :: ListN Bool 3 @-}
13 invalid :: [Bool]
14 invalid = [True, False]
```

LiquidHaskell Type Aliases

Type Aliases: used for reusability and readability.

```
1  -- with type alias
2  {-@ predicate Between Low High V = V >= Low && V <= High @-}
3  {-@ type Rng Low High = {v:Int | Between Low High v} @-}
4
5  -- we can use the predicate inline as well
6  {-@ grade :: Rng 0 100 -> {v:Int | Between 1 2 v} @-}
7  grade :: Int -> Int
8  grade score
9      | score >= 50 = 2
10     | otherwise   = 1
11
12 good = grade 85
13 bad = grade 120
```

LiquidHaskell Function Contracts

- **Precondition:** Constraint on the function's inputs.
- **Postcondition:** Constraint on the function's output.
- **Dependent types:** Output constraints can refer to input values.

```
1  {-@ type Rng Low High = {v:Int | v >= Low && v <= High} @-}
2
3  {-@ range :: lo:Int
4            -> hi:{Int | hi >= lo}
5            -> [Rng lo hi] @-}
6  range :: Int -> Int -> [Int]
7  range lo hi = [lo..hi]
8
9  -- [1..5] -> [1, 2, 3, 4, 5]
10 -- [5..1] -> []
```

LiquidHaskell Verification: Contracts

```
1  {-@ type Rng Low High = {v:Int | v >= Low && v <= High} @-}
2
3  {-@ range :: lo:Int
4            -> hi:{Int | lo <= hi}
5            -> [Rng lo hi] @-}
6  range :: Int -> Int -> [Int]
7  range lo hi = [lo..hi]
8
9  good = range 1 5
10 bad = range 5 1
```

- **SAFE:** range 1 5 satisfies the precondition $1 \leq 5$.
- **UNSAFE:** range 5 1 violates the precondition $5 \not\leq 1$.

LiquidHaskell Verification: Limitations

- Haskell's `[lo..hi]` is alias for `enumFromTo lo hi`.
- LiquidHaskell relies on refinement specifications to reason about library functions.
- Without a sufficiently precise specification for `enumFromTo`, it cannot prove that every element satisfies `Rng lo hi`.

```
1 -- Cannot verify the postcondition
2 range lo hi = [lo..hi]
```

Important

UNSAFE does not necessarily mean the code is wrong. It can mean LiquidHaskell cannot prove its correctness.

LiquidHaskell Verification: Recursive Range

```
1 {-@ range :: lo:Int
2           -> hi:{Int | lo <= hi}
3           -> [Rng lo hi] @-}
4 range :: Int -> Int -> [Int]
5 range lo hi
6   | lo <= hi = lo : range (lo + 1) hi
7   | otherwise = []
```

- At `lo == hi`, the recursive call is `range (hi + 1) hi`.
- Its precondition requires $hi + 1 \leq hi$, which is false.

LiquidHaskell Verification: Fixing the Boundary

```
1 {-@ range :: lo:Int
2           -> hi:{Int | lo <= hi}
3           -> [Rng lo hi] @-}
4 range :: Int -> Int -> [Int]
5 range lo hi
6   | lo < hi   = lo : range (lo + 1) hi
7   | otherwise = [lo]
```

- When `lo < hi`, integer arithmetic gives $lo + 1 \leq hi$.
- Otherwise, the precondition implies `lo == hi`; return the endpoint.
- Every returned element satisfies $lo \leq v \leq hi$.
- By default, LiquidHaskell tries to prove termination using a **decreasing argument**; here, lo increases instead.

LiquidHaskell Verification: Proving Termination

```
{-@ range :: lo:Int
          -> hi:{Int | lo <= hi}
          -> [Rng lo hi] / [hi - lo] @-}
range :: Int -> Int -> [Int]
range lo hi
  | lo < hi   = lo : range (lo + 1) hi
  | otherwise = [lo]
```

- `/ [hi - lo]` supplies an explicit **termination metric**.
- The precondition ensures $hi - lo \geq 0$.
- Each recursive call strictly decreases the metric:

$$hi - (lo + 1) = (hi - lo) - 1.$$

- At `lo == hi`, the metric is zero and recursion stops.